

โรงแรมอนันต์แบบกระจาย (Distributed Hilbert's Hotel) ด้วยเทคนิค Consistent Hashing

โจทย์ในสามบรรทัด

- เขียนโปรแกรมภาษา Python จัดแขกเข้าพักในเครือโรงแรมที่มีหลายอาคาร โดย “คำนวณ” ว่าแขกอยู่อาคารใดและห้องใด ไม่ใช่จัดบันทึกทีละคน
- เลือกอาคารด้วย Consistent Hashing และคำนวณหมายเลขห้องจากรหัสแขกโดยตรง เมื่อมีแขกใหม่เข้ามาแขกเดิมต้องไม่ย้ายห้อง
- เมื่อเพิ่มหรือลดอาคาร ต้องมีแขกย้ายน้อยที่สุด รายงานได้ว่าใครย้ายไปไหน แล้ววิเคราะห์ Big-O และทดลองเทียบกับวิธี $\text{hash mod } N$

1 โจทย์และขอบเขต

1.1 สถานการณ์

ในปัญหา Hilbert's Hotel เดิม โรงแรมมีห้องพักจำนวนอนันต์นับได้ และทุกครั้งที่แขกใหม่เข้ามาแขกเดิม “ทุกคน” ต้องย้ายห้อง คิดเป็นต้นทุน $\Theta(n)$ ต่อการรับแขกหนึ่งครั้ง ถูกต้องในเชิงคณิตศาสตร์ แต่ระบบจริงใช้ไม่ได้ ปีนี้โรงแรมขยายเป็น “เครือโรงแรม” ที่ประกอบด้วยอาคาร (node) จำนวน N หลัง แต่ละหลังยังมีห้องพักไม่จำกัดแขกแต่ละคนต้องถูกจัดเข้าอาคารด้วยการคำนวณจากรหัสของตัวเอง โจทย์ของโครงการนี้จึงไม่ใช่ “ย้ายแขกอย่างไรให้ครบ” แต่เป็น “ออกแบบอย่างไรให้แทบไม่ต้องย้ายเลย”

1.2 ทำไมวิธีที่ง่ายที่สุดจึงไม่พอ

ถ้าเลือกอาคารด้วย $\text{hash}(\text{รหัสแขก}) \bmod N$ การเพิ่มอาคารเพียงหลังเดียวทำให้ตัวหารเปลี่ยนจาก N เป็น $N+1$ ผลของการหารเอาเศษจึงเปลี่ยนเกือบทุกคน สรุปคือย้อนกลับไปเป็นปัญหาเดิมของ Hilbert's Hotel คือ “แขกเกือบทั้งหมดต้องย้าย”

1.3 Consistent Hashing แก้ปัญหานี้อย่างไร

จินตนาการวงแหวนที่มีตำแหน่งเรียงเป็นวงกลม นำทั้งอาคารและแขกไปวางบนวงแหวนด้วยค่าแฮชของตัวเอง
แขกแต่ละคนเข้าพักที่ “อาคารเจ้าของจุดแรกที่พบเมื่อเดินตามเข็มนาฬิกา”

ข้อดีอยู่ตรงนี้: เมื่อเพิ่มหรือถอนอาคารหนึ่งหลัง จะมีเพียงช่วงหนึ่งของวงแหวนที่เปลี่ยนเจ้าของ

แขกที่อยู่นอกช่วงนั้นไม่ได้รับผลกระทบเลย โดยเฉลี่ยจึงมีแขกย้ายราว K/N คน แทนที่จะเป็นเกือบทั้งหมด

ปัญหาที่ตามมาคือถ้าอาคารหนึ่งหลังมีจุดบนวงแหวนเพียงจุดเดียว ช่วงที่แต่ละอาคารรับผิดชอบจะยาวไม่เท่ากันมาก

โหลดจึงเบ้ ทางแก้คือ virtual node — แทนอาคารหนึ่งหลังด้วยจุดหลาย ๆ จุดกระจายทั่ววงแหวน ยิ่งจุดมาก

โหลดยิ่งสม่ำเสมอ แต่ก็ใช้หน่วยความจำเก็บวงแหวนมากขึ้น การหาจุดสมดุลของค่านี้เป็นส่วนหนึ่งของงาน

1.4 ขอบเขต: สิ่งที่ไม่ต้องทำ

- ไม่ต้องสร้างระบบเครือข่าย ระบบหลายเครื่อง หรือฐานข้อมูล อาคารแต่ละหลังคือวัตถุ (object) ภายในโปรแกรมเดียวที่รันบนเครื่องเดียว
 - ไม่ต้องจำลองเครื่องล้มจริงหรือการกู้คืนข้อมูล
 - ส่วนติดต่อแบบกราฟิกเป็นทางเลือก เมนูข้อความก็ได้คะแนนเต็ม
 - คำว่า “อนันต์” หมายถึงห้องพักไม่มีความจุสูงสุดที่กำหนดไว้ล่วงหน้า
- ไม่ต้องสร้างห้องว่างหรือแขกจำนวนอนันต์จริง
- ข้อมูลที่รับเข้าและประมวลผลในแต่ละครั้งมีจำนวนจำกัดตามทรัพยากรของเครื่อง

2 ผลลัพธ์การเรียนรู้

- 1) เลือกและอธิบายเหตุผลของโครงสร้างข้อมูลสำหรับวงแหวน ข้อมูลแขก และดัชนีค้นหา
โดยเชื่อมโยงกับรูปแบบการใช้งานจริง
- 2) ประยุกต์ hashing การเรียงลำดับ และการค้นหา พร้อมออกแบบคลาสที่มีหน้าที่ชัดเจน
- 3) วิเคราะห์ Big-O ของโค้ดที่เขียนจริง และแยก “จำนวนข้อมูลที่เปลี่ยนตำแหน่ง” ออกจาก
“เวลาที่ใช้ประมวลผล”

- 4) ออกแบบการทดลองที่ทำซ้ำได้ แล้วแปลผลเรื่องเวลา หน่วยความจำ ความสมดุลของโหลด และอัตราการย้ายแขก

3 นิยามและสัญลักษณ์

สัญลักษณ์	ความหมาย
N	จำนวนอาคารที่มีอยู่ก่อนการเพิ่มหรือลบอาคารในรอบนั้น โดย $N \geq 1$
V	จำนวน virtual node ต่ออาคาร ใช้เท่ากันทุกอาคารและคงที่ภายในการทดลองหนึ่งรอบ
M	จำนวนจุดบนวงแหวนทั้งหมด โดย $M = N \times V$
K	จำนวนแขกที่อยู่ในระบบ ณ เวลาที่วิเคราะห์ หรือก่อนเปลี่ยนจำนวนอาคาร
(c, s)	รหัสแขก: c คือหมายเลขช่องทางที่เดินทางมา และ s คือลำดับภายในช่องทางนั้น เป็นจำนวนเต็มบวก
(node_id, room_no)	ที่อยู่ห้อง: รหัสอาคาร และหมายเลขห้องภายในอาคาร

4 กติกาที่ต้องทำตาม

หัวข้อนี้คือข้อกำหนดที่ใช้ตรวจสอบให้คะแนนความถูกต้อง ส่วนที่เหลือของโจทย์เปิดให้ออกแบบเองได้

4.1 รหัสแขกและข้อมูลที่จัดเก็บ

- 1) แขกที่อยู่ในระบบต้องมีคู่ (c, s) ไม่ซ้ำกัน และรหัสอาคารต้องไม่ซ้ำและไม่เปลี่ยนตามจำนวนอาคาร
- 2) เก็บข้อมูลแขกและดัชนีค้นหาได้ แต่การเลือกอาคารต้องคำนวณจากรหัสแขกกับวงแหวนปัจจุบันได้เสมอ ห้ามอาศัยตารางคำตอบที่กำหนดไว้ล่วงหน้า
- 3) การเพิ่มแขกเป็นกลุ่มให้ระบุ c ลำดับเริ่มต้น s_start และจำนวนแขก จำนวนต้องเป็นจำนวนเต็มไม่ติดลบ (0 หมายถึงไม่เปลี่ยนข้อมูล) หากช่วงรหัสทับกับแขกที่มีอยู่ ให้ปฏิเสธทั้งกลุ่มและคงข้อมูลเดิมไว้

- 4) การนำแขกออกคือการลบข้อมูลการเข้าพัก ไม่ต้องสร้างห้องว่างทดแทน หากเพิ่มรหัสเดิมกลับเข้ามาอีกครั้ง ให้ถือเป็นการเข้าพักใหม่และคำนวณจากวงแหวนปัจจุบัน

4.2 หมายเลขห้อง

หมายเลขห้องต้องคำนวณจากรหัสแขกเท่านั้น ห้ามใช้วิธี “เลขห้องล่าสุดบวกหนึ่ง”

เพราะจะทำให้แขกใหม่ไปกระทบแขกเดิม ผลที่ตามมาคือเมื่อแขกย้ายอาคาร room_no ยังคงเดิม เปลี่ยนเฉพาะ node_id

ใช้ Cantor pairing function ต่อไปนี้ได้ทันที โดยกำหนด $a = c - 1$ และ $b = s - 1$

เพื่อแปลงรหัสให้เป็นจำนวนเต็มไม่ติดลบ

$$\text{room_no} = ((a + b) \times (a + b + 1)) // 2 + b + 1$$

เครื่องหมาย // คือการหารแบบจำนวนเต็ม ให้คำนวณด้วยจำนวนเต็มตลอด ห้ามแปลงเป็นทศนิยม

สูตรนี้ให้เลขห้องเป็นจำนวนเต็มบวกที่ไม่ซ้ำสำหรับรหัสแขกที่ต่างกัน กลุ่มที่ใช้สูตรนี้ต้องอธิบายได้ว่าทำไมจึงไม่ซ้ำ ไม่จำเป็นต้องคิดสูตรใหม่

รหัสแขก (c, s)	(1, 1)	(2, 1)	(1, 2)	(3, 1)	(2, 2)
room_no	1	2	3	4	5

กลุ่มที่อยากใช้วิธีอื่น เช่น กำลังของจำนวนเฉพาะ หรือ pairing function แบบอื่น ทำได้

แต่ต้องพิสูจน์ความไม่ซ้ำในรายงานและแสดงตัวอย่างเทียบเคียงกับตารางข้างต้น

4.3 วงแหวนและการเลือกอาคาร

- 1) ใช้วงแหวนตำแหน่งจำนวนเต็ม 0 ถึง $2^{64} - 1$ เลือกแฮชที่ให้ผลคงที่ข้ามการเปิดโปรแกรม เช่น SHA-256

แล้วตัดมาเป็นตำแหน่ง 64 บิต ระบุการเข้ารหัสข้อความและวิธีแปลงค่าในรายงาน ห้ามใช้ hash() ของ Python เป็นแฮชหลัก เพราะค่าไม่คงที่ข้ามการรัน

- 2) แยกข้อความที่นำไปแฮชระหว่างแขกกับ virtual node ให้ชัดเจน เช่น guest:c:s และ node:node_id:j โดย $j = 0$ ถึง $V - 1$ และใช้รูปแบบเดิมทุกครั้ง

- 3) เลือก virtual node จุดแรกที่มีตำแหน่งมากกว่าหรือเท่ากับตำแหน่งของแขก หากไม่มี
ให้วนกลับไปจุดแรกของวงแหวน แล้วใช้รหัสอาคารเจ้าของจุดนั้น
- 4) หากจุดหลายจุดเหมาะสมกัน ให้เก็บไว้ทุกจุดและเรียงด้วยคีย์ (ตำแหน่ง, node_id, j)
เมื่อแขกตรงตำแหน่งนั้นให้ใช้จุดแรกตามลำดับนี้ แขกหลายคนเหมาะสมตรงกันได้ และยังมีเลขห้องต่างกัน

4.4 การเพิ่ม/ลบอาคาร และการนับการย้าย

- 1) เพิ่มอาคารโดยเพิ่มเฉพาะ virtual node ของอาคารใหม่ ลบอาคารโดยลบ virtual node ของอาคารนั้นทั้งหมด
ห้ามเปลี่ยนรหัสหรือตำแหน่งของจุดที่ยังอยู่ และห้ามเปลี่ยนค่า V ระหว่างรอบเดียวกัน
- 2) เมื่อเพิ่มอาคาร แขกที่เปลี่ยนอาคารต้องย้ายเข้าอาคารใหม่เท่านั้น เมื่อถอนอาคาร แขกในอาคารอื่นต้องไม่ย้าย
หากผลไม่เป็นเช่นนั้นแสดงว่ากลไกยังไม่ถูกต้อง
- 3) จำนวนแขกที่ย้าย คือจำนวนรหัสแขกที่ node_id ก่อนและหลังคำสั่งต่างกัน นับคนละหนึ่งครั้ง ห้ามใช้จำนวน
virtual node ที่เปลี่ยนแทนจำนวนแขก รายการย้ายต้องระบุรหัสแขก อาคารเดิม อาคารใหม่
และหมายเลขห้อง
- 4) รายงานอัตราการย้ายเป็น จำนวนแขกที่ย้าย / K เมื่อ $K > 0$ หาก $K = 0$ ให้รายงานจำนวนย้ายเป็น 0
และระบุว่าไม่คำนวณอัตราเพราะไม่มีแขก

ค่าคาดหวังที่ควรเห็น

เมื่อแขกกระจายสม่ำเสมอ การเพิ่มจาก N เป็น $N + 1$ อาคารมีจำนวนย้ายคาดหวังราว $K/(N + 1)$

และการถอนอาคารหนึ่งหลังจาก N อาคารมีค่าคาดหวังราว K/N

นี่เป็นค่าคาดหวัง ไม่ใช่ขอบเขตกรณีเลวร้ายที่สุด ผลจริงขึ้นกับตำแหน่งบนวงแหวนและชุดข้อมูล

จึงไม่ต้องได้เท่าค่านี้นี้ทุกครั้ง สิ่งที่ต้องทำคือรายงานค่าที่วัดได้จริงและอธิบายว่าทำไมจึงต่างจากค่าคาดหวัง

4.5 กรณีขอบที่ต้องจัดการ

“กรณีขอบ” คือข้อมูลนำเข้าที่อยู่ตรงเส้นแบ่งของความถูกต้อง เช่น ค่าว่าง เหลือชิ้นเดียว ซ้ำ หรือไม่มีอยู่จริง ตรงข้ามกับกรณีปกติที่มักถูกทดสอบอยู่แล้ว โปรแกรมที่เขียนตรงไปตรงมาจะไม่ถูกต้องในจุดเหล่านี้

ทุกกรณีต่อไปนี้จะต้องแจ้งเหตุผล คงข้อมูลเดิมไว้ และโปรแกรมต้องไม่หยุดทำงาน

กรณี	ถ้าไม่ดักไว้จะเกิดอะไร
เพิ่มรหัสอาคารที่มีอยู่แล้ว	จุด virtual node ของอาคารเดิมถูกใส่ซ้ำลงวงแหวน นับโหนดเพี้ยน และตอนลบทีหลังจะลบออกไม่หมด
ลบรหัสอาคารที่ไม่มีอยู่	เกิด KeyError หรือไม่ได้ลบอะไรเลย แต่ยังรายงานจำนวนคนย้ายออกมา
ลบอาคารสุดท้าย (แม้ไม่มีแขก)	วงแหวนกลายเป็นว่าง การค้นหาครั้งถัดไปจะใช้งานไม่ได้หรือวนไม่จบ และขัดกับนิยาม $N \geq 1$ ในหัวข้อ 3
เพิ่มรหัสแขกที่ซ้ำ	รหัส (c, s) เดิมให้ room_no เดิม กลายเป็นสองคนห้องเดียวกัน หรือเขียนทับข้อมูลคนเก่าโดยไม่แจ้ง
นำแขกที่ไม่มีอยู่ออก	เกิด KeyError หรือหักจำนวนแขกทั้งที่ไม่ได้ลบใคร ทำให้สถิติผิดตามไปทั้งหมด
ค้นหาแล้วไม่พบ	คืนค่าว่างแล้วไปพึ่งตอนนำไปแสดงผล แทนที่จะบอกผู้ใช้งานว่า “ไม่พบ”
ยังไม่มีแขกในระบบ ($K = 0$)	หาค่าเฉลี่ยด้วยการหารด้วยศูนย์ ส่วน CV และอัตราการย้ายคำนวณไม่ได้ (ดูติกาในหัวข้อ 4.4 ข้อ 4)

จุดรวมของทุกกรณีคือ “ปฏิเสธอย่างสุภาพ” ไม่ใช่ทำให้โปรแกรมใช้งานไม่ได้ ด้วยเหตุผลเดียวกันนี้ หัวข้อ 4.1 ข้อ 3 จึงสั่งให้ปฏิเสธการเพิ่มแขกเป็นกลุ่มทั้งกลุ่ม หากมีรหัสซ้ำแม้เพียงรายการเดียว

4.6 ไบเบรารีที่ใช้ได้

ใช้ list, dict, set, hashlib และ bisect ได้ แต่ต้องเขียนตรรกะวงแหวน การเลือกอาคาร และการย้ายเอง ห้ามใช้ไลบรารี Consistent Hashing สำเร็จรูป

4.7 การหาแขกที่ต้องย้าย

เมื่อเพิ่มหรือลบอาคาร โปรแกรมต้องหาให้ได้ว่า “ใครบ้างที่ต้องย้าย” ทำได้สองแบบ

ทั้งสองแบบให้ผลลัพธ์เหมือนกันทุกประการ ต่างกันที่ปริมาณงานที่ใช้ไปกว่าจะได้คำตอบนั้นมา

แบบ ก — ตรวจใหม่ทุกคน วนดูแขกทั้ง K คน คำนวณอาคารใหม่ให้ทุกคน แล้วเทียบกับอาคารเดิม

ใครเปลี่ยนคือคนที่ย้าย เขียนง่ายและถูกต้อง แต่ต้องแตะแขกทุกคน แม้จะมีคนย้ายจริงเพียงไม่กี่คน

แบบ ข — หาเฉพาะช่วงที่ได้รับผลกระทบ ดูว่า virtual node

ของอาคารที่เพิ่มหรือลบครอบคลุมช่วงใดบนวงแหวน แล้วดึงเฉพาะแขกที่ตกอยู่ในช่วงนั้น

ต้องเก็บแขกแบบเรียงตามตำแหน่งแฮชเพิ่มเข้ามา เขียนยากกว่า แต่ทำงานน้อยกว่ามาก

เลือกแบบ ก ได้ ไม่เสียคะแนนความถูกต้อง แต่ในรายงานต้องเขียน Big-O ของแบบที่เขียนจริง

ห้ามนำตัวเลขของแบบ ข มาใส่ ส่วนกลุ่มที่ทำแบบ ข จะได้เปรียบในหัวข้อการวิเคราะห์

5 ฟังก์ชันที่โปรแกรมต้องมี

1) เริ่มระบบด้วยจำนวนอาคาร N ค่า V จำนวนช่องทาง และจำนวนแขกของแต่ละช่องทาง

โดยลำดับของแต่ละช่องทางเริ่มที่ 1

2) เพิ่มแขกรายบุคคลหรือเป็นกลุ่ม ทั้งช่องทางเดิมและช่องทางใหม่

โดยที่อยู่ห้องของแขกเดิมต้องไม่เปลี่ยนหากวงแหวนไม่เปลี่ยน

3) นำแขกรายบุคคลออกด้วยรหัส (c, s) หากไม่พบให้แจ้งผลโดยไม่กระทบข้อมูลอื่น

4) เพิ่มหรือลบอาคารตามรหัสที่ผู้ใช้ระบุ พร้อมรายงานจำนวน อัตราส่วน และรายการแขกที่ย้าย ตามกติกาในหัวข้อ

4.4

5) ค้นหาสองทาง: ค้นด้วยรหัสแขกเพื่อดูอาคารและหมายเลขห้อง และค้นด้วย (node_id, room_no)

เพื่อดูรหัสแขก หากไม่พบต้องแจ้งชัดเจน

- 6) แสดงรายการห้องที่มีผู้พัก เรียงตาม node_id จากน้อยไปมาก และเรียง room_no แบบตัวเลขจากน้อยไปมากภายในแต่ละอาคาร
- 7) รายงานความสมดุลของโหนด: จำนวนแขกทุกอาคาร (รวมอาคารที่มี 0 คน) ค่าสูงสุด ต่ำสุด เฉลี่ย และส่วนเบี่ยงเบนมาตรฐาน
- 8) วัดเวลาของฟังก์ชันหลักและหน่วยความจำที่ใช้ ตามวิธีในหัวข้อ 7 พร้อมแสดงพารามิเตอร์ของการทดลอง
- 9) เปรียบเทียบ Consistent Hashing กับวิธี hash mod N โดยใช้ชุดแขกเดียวกันและการเปลี่ยนอาคารแบบเดียวกัน
- 10) ส่งออกไฟล์ CSV แบบ UTF-8 อย่างน้อยคอลัมน์ channel_id, sequence_id, node_id, room_no รวมถึงส่งออกการย้ายและผลการทดลองได้

ข้อมูลจำนวนมากจะแสดงบนหน้าจอเพียงบางส่วนก็ได้ แต่ต้องบอกจำนวนทั้งหมด ส่วนไฟล์ส่งออกต้องครบ การอ่านข้อมูลกลับจากไฟล์ GUI และภาพเคลื่อนไหว ไม่ใช้งานบังคับ

6 การออกแบบเชิงวัตถุและการแบ่งงาน

- แยกตรรกะหลักออกจากเมนูและการแสดงผล ออกแบบคลาสตามหน้าที่ เช่น Guest, HashRing และ HotelSystem
- ชื่อและจำนวนคลาสปรับได้ตามเหตุผลของกลุ่ม ไม่บังคับใช้ inheritance หากไม่เหมาะกับแบบจำลอง
- รายงานต้องมีแผนภาพคลาสอย่างง่าย หน้าที่ของแต่ละคลาส โครงสร้างข้อมูลที่ใช้ และตัวอย่างอย่างน้อยหนึ่งกรณีที่ข้อมูลหลายส่วนต้องปรับให้สอดคล้องกัน เช่น เมื่อแขกย้ายอาคารแล้วดัชนีค้นหาห้องยังให้คำตอบถูกต้อง
- สมาชิกทุกคนต้องมีส่วนเขียนโปรแกรมและทดสอบ
- แนวทางแบ่งงาน: วางแผนและการเลือกอาคาร / ข้อมูลแขกและเลขห้อง / การเพิ่ม-ลบอาคารและการย้าย / การทดลองและกราฟ / ส่วนติดต่อผู้ใช้และไฟล์ ฯ
- ตกลงรูปแบบข้อมูลและวิธีเรียกใช้ร่วมกันก่อนเริ่มเขียน และแต่ละคนต้องอธิบายงานของตนกับกติกาลหลักของระบบได้

7 การทดลองที่ต้องทำ

7.1 ชุดการทดลอง

การทดลอง	ค่าที่เปลี่ยน	ค่าที่กำหนด	ผลที่ต้องเปรียบเทียบ
A ขนาดข้อมูล	$K = 1,000; 10,000; 100,000$	$N = 10, V = 32$	เวลาจัดแขก ค้นหา เรียงลำดับ และหน่วยความจำ
B จำนวนอาคาร	$N = 5; 10; 20$	$K = 10,000, V = 32$	เพิ่ม 1 อาคารและลบ 1 อาคาร: จำนวนย้าย อัตราย้าย และเวลา เทียบกับ $\text{hash mod } N$
C จำนวนจุดเสมือน	$V = 1; 8; 32$	$N = 10, K = 10,000$	จำนวนแขกต่ออาคาร ความสมดุล หน่วยความจำ และการย้าย

สร้างแขกจาก 10 ช่องทาง กระจายจำนวนให้ใกล้เคียงกัน แต่ละคู่เปรียบเทียบต้องใช้รหัสแขก รหัสอาคาร และค่าแฮชชุดเดียวกัน ใช้เครื่องและ Python รุ่นเดียวกันตลอด ทำการทดลองละ 3 รอบ โดยเปลี่ยน salt ที่ระบุไว้ชัดเจน (เช่น 0, 1, 2) และใช้ salt เดียวกันกับทั้งสองวิธีในรอบเดียวกัน บันทึกค่าที่ทำให้สร้างผลซ้ำได้ (salt คือ ข้อความสั้น ๆ ที่เติมเข้าไปในสตริงก่อนนำไปแฮช เพื่อให้ได้ตำแหน่งบนวงแหวนชุดใหม่ ทั้งที่ข้อมูลแขกและอาคารยังเป็นชุดเดิม)

สำหรับ $\text{hash mod } N$ ให้เก็บรายการรหัสอาคารตามลำดับที่เพิ่ม แล้วเลือกอาคารที่ดัชนี $H(\text{guest}) \bmod N$ เมื่อเพิ่มให้ต่อท้ายรายการ เมื่อลบให้เอารหัสนั้นออกโดยรักษาลำดับที่เหลือ และเปรียบเทียบการย้ายด้วยรหัสอาคารจริง ไม่ใช่เทียบเพียงดัชนี

7.2 วิธีวัดและการรายงาน

เวลา ใช้ `time.perf_counter()` แยกเวลาสร้างวงแหวน จัดแขก ค้นหา เรียงลำดับ และเพิ่ม/ลบอาคาร

ไม่รวมเวลารับแป้นพิมพ์ แสดงผล และเขียนไฟล์ วัดการค้นหาเป็นชุด 1,000 ครั้งแล้วหารต่อครั้ง

รายงานค่ามัธยฐานของ 3 รอบ

หน่วยความจำ ใช้ tracemalloc วัดส่วนที่เพิ่มจากจุดเริ่มต้นเดียวกัน รายงาน current และ peak เป็น MiB พร้อมระบุว่าหน่วยความจำที่ Python ติดตาม ไม่ใช่ของทั้งโปรเซส และทำรอบวัดหน่วยความจำแยกจากรอบจับเวลา

ความสมดุล ใช้ส่วนเบี่ยงเบนมาตรฐานหารด้วยค่าเฉลี่ย ได้เป็นค่า CV ค่ายิ่งต่ำยิ่งสมดุล หากไม่มีแยกให้แสดงสถิติพื้นฐานเป็น 0 และไม่คำนวณ CV

ส่งกราฟอย่างน้อย 3 ภาพ: (ก) เวลาเทียบ K, (ข) อัตราการย้ายเทียบ N ของทั้งสองวิธี, (ค) ผลของ V ต่อ CV

และหน่วยความจำ ทุกกราฟต้องระบุหน่วยและพารามิเตอร์ พร้อมบอกความแปรปรวน 3 รอบ เช่น

ค่าเฉลี่ยกับช่วงต่ำสุด-สูงสุด หากเครื่องทำชุดใหญ่ไม่ไหว ให้เก็บหลักฐานและรายงานขอบเขตที่ทำได้

8 การวิเคราะห์ Big-O ในรายงาน

วิเคราะห์จากโค้ดที่ส่งจริง โดยใช้ N , V , $M = N \times V$ และ K ตามนิยาม และระบุทุกครั้งว่าค่าที่ให้เป็นกรณีเลวร้าย

ที่สุด ค่าเฉลี่ย หรือค่าคาดหวัง หากใช้โครงสร้างข้อมูลหลายชุด ต้องรวมต้นทุนการปรับให้สอดคล้องกันทุกชุดด้วย

- 1) สร้างวงแหวนเริ่มต้น และจัดแยกเริ่มต้น (แยกสองขั้นตอน)
- 2) เพิ่มแขกหนึ่งคน เพิ่มแขกเป็นกลุ่ม และนำแขกหนึ่งคนออก
- 3) ค้นหาผ่านรหัสแยก ค้นหาผ่านที่อยู่ห้อง และคำนวณอาคารจากวงแหวนโดยไม่ใช้ผลที่บันทึกไว้
- 4) เรียงรายการห้อง รวมต้นทุนการสร้างรายการผลลัพธ์ (ถ้ามี)
- 5) เพิ่มและลบอาคาร โดยแยก (ก) การปรับวงแหวน (ข) การหาแขกที่ได้รับผลกระทบ (ค) การปรับข้อมูลและดัชนีของแขกที่ย้าย
- 6) พื้นที่ของวงแหวน ข้อมูลแยก ดัชนีเพิ่มเติม และพื้นที่ชั่วคราว เช่น สำเนาถาวรย้ายหรือรายการผลลัพธ์

8.1 แนวทางประกอบการวิเคราะห์

ประเด็น	แนวทางพิจารณา
ค้นหาบนวงแหวน	ถ้าเก็บ M จุดแบบเรียงลำดับและใช้ binary search จะใช้ $O(\log M)$ เฉพาะขั้นตอนหาจุด ไม่ใช่ต้นทุนของทุกฟังก์ชัน

ประเด็น	แนวทางพิจารณา
แทรกจุดลงใน list	การหาตำแหน่งใช้ $O(\log M)$ แต่การเลื่อนสมาชิกใน list อาจใช้ $O(M)$ ต่อหนึ่งจุด ถ้าเพิ่มทีละ V จุดต้องวิเคราะห์ตามวิธีที่เขียนจริง
หาแขกที่ต้องย้าย	ถ้าตรวจแขกครบ K คนและคั่นวงแหวนให้ทุกคน ขั้นตอนนี้มีต้นทุน $O(K \log M)$ แม้จะมีคนย้ายจริงเพียงไม่กี่คน (ดูการเทียบสองวิธีในหัวข้อ 4.7)
จำนวนแขกที่ย้าย	เป็นจำนวนข้อมูลที่เปลี่ยนอาคาร ไม่ใช่เวลาทั้งหมดของคำสั่ง และอาจเป็น 0 ในชุดข้อมูลขนาดเล็ก
หน่วยความจำ	วงแหวนมี $O(NV)$ จุด หากเก็บข้อมูลแขกและดัชนีคงที่ต่อคน พื้นที่ข้อมูลหลักจะเป็น $O(NV + K)$ ภายใต้สมมติฐานว่าขนาดรหัสคงที่

ข้อผิดพลาดที่พบบ่อยสองข้อ

- สรุปว่า hash mod N ใช้พื้นที่ $O(1)$ ทั้งที่โปรแกรมยังเก็บรายการรหัสอาคาร N รายการและข้อมูลแขก K คน ที่เป็น $O(1)$ คือพื้นที่เสริมของสูตรคำนวณตำแหน่งเท่านั้น
- สันนิษฐานสมมติฐานว่าขนาดของรหัสและการเปรียบเทียบถือว่าคงที่ ทั้งที่ Python ใช้จำนวนเต็มที่ขยายขนาดได้ ต้นทุนของเลขห้องหรือแฮชจึงอาจโตตามความยาวรหัส

8.2 ตารางเปรียบเทียบที่ต้องสรุปเอง

เปรียบเทียบ hash mod N กับ Consistent Hashing ในหัวข้อ: การคำนวณอาคาร, การเพิ่ม/ลบอาคาร, อัตราการย้าย, พื้นที่จัดเส้นทาง, พื้นที่ข้อมูลรวม และความสมดุลของโหลด โดยแยก “ค่าจากการวิเคราะห์” ออกจาก “ค่าที่วัดได้จริง” และไม่สรุปว่าผลต้องตรงกับทฤษฎีทุกครั้ง

9 การทดสอบความถูกต้อง

9.1 ตัวอย่างวงแหวนเล็กสำหรับตรวจตรรกะ

ตัวอย่างนี้ใช้วงแหวนขนาด 100 และ $V = 1$ โดยกำหนดตำแหน่งไว้วงหน้าเพื่อตรวจตรรกะเท่านั้น ไม่ใช่ผลจาก SHA-256 ที่ต้องทำให้ตรง วางอาคาร A ที่ 20, B ที่ 50 และ C ที่ 80 แล้วค้นหาด้วยกติกา “มากกว่าหรือเท่ากับ” และวนกลับต้นวงแหวน

ตำแหน่งแฮชของแขก	เริ่มต้น A B C	เพิ่ม D ที่ 35	ลบ B จากสถานะเริ่มต้น
10	A	A	A
20	A	A	A
25	B	D	C
35	B	D	C
50	B	B	C
70	C	C	C
90	A	A	A

ให้แต่ละแถวเป็นแขกคนละคน รวม 7 คน เมื่อเพิ่ม D ต้องมีคนย้าย 2 คน และเมื่อลบ B

จากสถานะเริ่มต้นต้องมีคนย้าย 3 คน แต่ละกรณีเริ่มจากสถานะ A B C ใหม่

และหมายเลขห้องของผู้ย้ายทุกคนต้องคงเดิม สังเกตว่าแขกที่ตำแหน่ง 90 วนกลับไปหาอาคาร A

9.2 รายการทดสอบขั้นต่ำ

- 1) เพิ่มแขกใหม่ทั้งช่องทางเดิมและช่องทางใหม่ แล้วตรวจว่าที่อยู่ห้องของแขกเดิมไม่เปลี่ยนแม้แต่คนเดียว
- 2) ตรวจเลขห้องตามตัวอย่างสูตร และตรวจว่ารหัสแขกที่ต่างกันไม่ทำให้ที่อยู่ห้องซ้ำกัน
- 3) ปฏิเสธรหัสแขกซ้ำ รวมกรณีเพิ่มเป็นกลุ่มที่ซ้ำเพียงบางรายการ โดยข้อมูลเดิมต้องไม่เปลี่ยน

4) ค้นหาตำแหน่งตรงจุดอาคารพอดี ระหว่างจุด และหลังจุดสุดท้าย (กรณีวนกลับต้นวงแหวน) รวมถึงกรณี virtual node ซนกันด้วยข้อมูลจำลอง

5) เพิ่มและลบอาคารเมื่อ $V > 1$ แล้วตรวจว่าผู้ย้ายเป็นไปตามกติกา จำนวนแซกรวมไม่หายไปไม่เพิ่ม และจำนวนรายการย้ายตรงกับยอดที่รายงาน

6) กรณีชอบตามหัวข้อ 4.5 ทั้งหมด และตรวจว่าจำนวนแถวในไฟล์ CSV ตรงกับจำนวนแซกปัจจุบัน ต้องส่งสคริปต์ทดสอบอัตโนมัติสำหรับกติกาหลัก ใช้ assert หรือ unittest ได้

พร้อมรายงานผลผ่าน/ไม่ผ่านและคำอธิบายกรณีที่ยังไม่ผ่าน ภาพหน้าจออย่างเดียวนับเป็นหลักฐานความถูกต้อง

10 แผนงานและการส่งงาน

กลุ่มละ 4–5 คน จับกลุ่มตามอิสระ แจ้งรายชื่อภายในวันพฤหัสบดีที่ 24 กันยายน 2569

10.1 สิ่งที่ต้องส่งผ่าน MS Teams

1) โค้ด Python ทั้งหมด รวมไฟล์เริ่มโปรแกรมและสคริปต์ทดสอบ พร้อม README ที่ระบุรุ่น Python วิธีเรียกโปรแกรม และวิธีรันทดสอบ

2) รายงานไฟล์ Word และ PDF เนื้อหาตรงกัน ครอบคลุมแบบข้อมูลและคลาส โครงสร้างข้อมูล กติกาการคำนวณ Big-O วิธีทดลอง ตารางและกราฟ ผลทดสอบ ข้อจำกัด และแหล่งอ้างอิง

3) ไฟล์ CSV ได้แก่ ข้อมูลแซกตัวอย่าง รายการย้าย และข้อมูลผลทดลองที่ใช้สร้างกราฟ พร้อมพารามิเตอร์และ salt ของแต่ละรอบ

4) ตารางหน้าที่สมาชิก ระบุส่วนที่แต่ละคนพัฒนาและทดสอบ (ใส่ไว้ในรายงานได้)

ส่งเป็นไฟล์ ZIP หนึ่งชุดที่เปิดแล้วพบ README ทันที ไม่ต้องแนบ virtual environment หรือแพ็คเกจที่ติดตั้งไว้

10.2 กำหนดส่งและการนำเสนอ

ส่งงานและนำเสนอภายในจะประกาศอีกครั้ง

10.3 งานเพิ่มเติม (ไม่บังคับ)

เช่น การวาดภาพวงแหวน การอ่านข้อมูลกลับจากไฟล์

หรือการหาเฉพาะแยกในช่วงที่ได้รับผลกระทบแทนการตรวจทุกคน ให้ทำหลังจากฟังก์ชันบังคับผ่านการทดสอบแล้ว
งานเพิ่มเติมไม่ใช่เงื่อนไขของคะแนนเต็ม และไม่ใช้ชดเชยข้อผิดพลาดของกติกาหลัก

11 เกณฑ์การให้คะแนน

คะแนนเต็ม 30 คะแนน การประเมินประสิทธิภาพพิจารณาจากวิธีวัด การวิเคราะห์ และข้อแลกเปลี่ยนที่อธิบายได้
ไม่ได้จัดอันดับจากความเร็วของเครื่องแต่ละกลุ่ม

ในรายงานที่ส่งให้แจ้งสัดส่วนร้อยละของการทำงานของสมาชิกแต่ละคน พร้อมลงลายมือชื่อแต่ละคน

เกณฑ์	คะแนน	หลักฐานที่พิจารณา
ความถูกต้องและการทดสอบ	9	จัดห้องและค้นหาถูกต้อง ไม่ซ้ำไม่หาย รักษากติกาการย้าย จัดการกรณีขอบครบ และมีสคริปต์ทดสอบที่รันซ้ำได้
โครงสร้างข้อมูลและ OOP	6	เลือกโครงสร้างข้อมูลอย่างมีเหตุผล คลาสมีหน้าที่ชัดเจน แยกตรรกะออกจากส่วนติดต่อ และรักษาข้อมูลกับดัชนีให้สอดคล้องกัน
การวิเคราะห์ Big-O	5	วิเคราะห์ครบและตรงกับโค้ดจริง แยกต้นทุนแต่ละขั้นตอน นับพื้นที่เพิ่มเติม และระบุสมมติฐาน
การทดลองและการอธิบายผล	5	เปรียบเทียบอย่างเป็นธรรม ทำซ้ำได้ ครอบคลุมเวลา หน่วยความจำ โหลด และการย้าย พร้อมอธิบายผลที่ต่างจากค่าคาดหวัง
รายงานและการสาธิต	5	รายงานอ่านเข้าใจ กราฟมีหน่วย อ้างอิงเหมาะสม โปรแกรมเรียกใช้ได้จริง และสาธิตครบประเด็นหลัก
รวม	30	

แต่ละเกณฑ์ให้คะแนนตามสัดส่วนของหลักฐานที่ทำได้จริง หลักฐานครบและถูกต้องได้เต็ม ทำได้บางส่วนได้ตามส่วนที่ตรวจสอบได้ ส่วนที่ไม่มีหลักฐานหรืออธิบายไม่ได้อาจไม่ได้คะแนน คะแนนของสมาชิกในกลุ่มเดียวกันจึงอาจต่างกัน

12 เช็กลิสต์ก่อนส่ง

- เพิ่มแขกใหม่แล้ว ที่อยู่ห้องของแขกเดิมไม่เปลี่ยนแม้แต่คนเดียว
- เลขห้องคำนวณจากรหัสแขกถ้วน ๆ ไม่มีที่ไหนใช้ “เลขล่าสุด + 1”
- เพิ่มอาคารแล้วผู้ย้ายเข้าอาคารใหม่เท่านั้น ลบอาคารแล้วแขกอาคารอื่นไม่ขยับ
- จำนวนที่รายงานว่าย้าย เท่ากับจำนวนรายการย้ายที่พิมพ์ออกมาจริง
- ปิดโปรแกรมแล้วเปิดใหม่ ค่าแฮชยังเหมือนเดิม (ไม่ได้ใช้ hash() ของ Python)
- กรณีขอบทั้งหมดในหัวข้อ 4.5 ไม่ทำให้โปรแกรมหยุดทำงาน
- จำนวนแถวใน CSV เท่ากับจำนวนแขกในระบบ
- กราฟทุกภาพมีชื่อแกน หน่วย และพารามิเตอร์กำกับ
- ตัวเลข Big-O ในรายงานตรงกับโค้ดที่ส่งจริง ไม่ใช่ค่ามาตรฐานที่ลอกมา

13 แหล่งศึกษาประกอบ

ระบุแหล่งที่นำมาใช้จริงในรายงาน และอธิบายการนำไปใช้ด้วยความเข้าใจของตนเอง

- ทบทวนปัญหา Hilbert's Hotel: <https://youtu.be/HLTjDXT9SqQ>
- Consistent Hashing โดย ByteByteGo: <https://www.youtube.com/watch?v=UF9lqmg94tk>
- Python bisect (ต้นทุนการค้นหาและการแทรกในรายการ):
<https://docs.python.org/3/library/bisect.html>
- Python tracemalloc (การวัดหน่วยความจำ): <https://docs.python.org/3/library/tracemalloc.html>
- Python time.perf_counter (การจับเวลา):
https://docs.python.org/3/library/time.html#time.perf_counter